

KEYWORD → CODE. BOTH TRACKS.

ALWAYS TYPE FIRST (every exercise)

```
import os
import numpy as np
import imageio.v3 as iio
import scipy.ndimage as ndi
from matplotlib import pyplot as plt
from skimage import color

root = r"C:\Users\Elika\Desktop\IVPAD"
input_root = os.path.join(root, "Images")
output_root = os.path.join(root, "ID_Surname_Name_Outputs")
if not os.path.exists(output_root):
    os.makedirs(output_root)

img = iio.imread(os.path.join(input_root, "FILENAME"))
print(img.shape, img.dtype, img.min(), img.max())
```

Extra imports — only when needed:

```
from skimage.transform import rescale, resize          # scaling
from skimage.segmentation import watershed, mark_boundaries # watershed
from skimage.feature import peak_local_max            # watershed
from skimage.color import label2rgb                   # watershed
import torch, torch.nn as nn, torch.optim as optim    # Ex 5, 6
```

"normalised float32 in [0,1]"

```
img = img.astype(np.float32) / 255.0
```

"histogram"

```
n_org, _ = np.histogram(img, np.arange(257))
plt.bar(np.arange(256), n_org)
```

257 for histogram, 256 for bar. Needs uint8, not float.

"Full-Scale Histogram Stretching"

```
img_fshs = img.astype(np.float32)
img_fshs = (img_fshs - img_fshs.min()) / (img_fshs.max() - img_fshs.min()) * 255.0
img_fshs = np.clip(img_fshs, 0, 255)
img_fshs = img_fshs.astype(np.uint8)
```

"translation" / "+60 rows, -100 columns" (A only)

```
img_translated = ndi.shift(img_fshs, (60, -100))
```

"rotation, 45°, nearest-neighbour and bilinear" (B only)

```
img_rot_nn = ndi.rotate(img, 45, order=0)
img_rot_bl = ndi.rotate(img, 45, order=1)
```

Shape changes → no sharex/sharey with the original.

"upscale by a factor of 3, NN and bilinear" (B only)

```
img_norm = img_fshs.astype(np.float32) / 255.0
img_nn = rescale(img_norm, scale=3, order=0)
img_bl = rescale(img_norm, scale=3, order=1)
```

Normalise first or one panel is black.

"back to the original size"

```
img_back = resize(img_small, img_norm.shape, order=1)
```

"Gaussian filter with $\sigma = 3$ " (A: $\sigma=3$ · watershed: $\sigma=2$)

```
img_gauss = ndi.gaussian_filter(img, 3)
```

Second arg = sigma.

"box filter (built manually as a kernel)" (A: 15×15 · B: 9×9)

```
side = 9
box_filter = np.ones((side, side), dtype=np.float32) / (side * side)
img_box = ndi.correlate(img, box_filter)
```

Second arg = size. Forget the division → white image.

"median filter with window size 7" (B only)

```
img_median = ndi.median_filter(img, 7)
```

"negative transformation" (A only)

```
img_negative = 1.0 - img
```

"Gamma Correction" (A: 0.4, 2.5 · B: 0.3, 3.0)

```
img = img.astype(np.float32) / 255.0
img_gamma_04 = img ** 0.4
img_gamma_25 = img ** 2.5
```

No clip needed.

"Unsharp Masking" (A only)

```
side = 15
box_filter = np.ones((side, side), dtype=np.float32) / (side * side)
img_blur = ndi.correlate(img, box_filter)
g_mask = img - img_blur
img_um = np.clip(img + 1.0 * g_mask, 0, 1)
```

Mask is signed → display with NO clim.

"Laplacian Sharpening (with diagonals)" (A only)

```
laplacian = np.array([[ -1, -1, -1],
                      [-1,  8, -1],
                      [-1, -1, -1]], dtype=np.float32)
img_lapl = ndi.correlate(img, laplacian)
img_ls = np.clip(img + 0.15 * img_lapl, 0, 1)
```

Positive centre → PLUS. Looks blurry = wrong sign.

"Sobel" / "gradient magnitude"

```
sobel_x = np.array([[ -1,  0,  1],
                    [-2,  0,  2],
                    [-1,  0,  1]], dtype=np.float32)
sobel_y = sobel_x.T
img_dx = ndi.correlate(img_blur, sobel_x)
img_dy = ndi.correlate(img_blur, sobel_y)
img_grad = np.sqrt(img_dx**2 + img_dy**2)
```

"and phase" (A only)

```
img_phase = np.arctan(img_dx / img_dy)
```

"Watershed"

```
coords = peak_local_max(-img_grad, min_distance=30)
markers = np.zeros(img_grad.shape, dtype=int)
markers[tuple(coords.T)] = np.arange(1, len(coords) + 1)
labels = watershed(img_grad, markers=markers)

overlay = label2rgb(labels, image=img, bg_label=0, alpha=0.4)
boundaries = mark_boundaries(img, labels)
print(labels.max())      # should be TENS
```

If the marker line won't come:

```
for i in range(len(coords)):
    markers[coords[i,0], coords[i,1]] = i + 1
```

Overlay = colour → no cmap, no clim.

"extract the R, G, B channels"

```
H, W, C = img.shape
img_red = img[:, :, 0]
img_green = img[:, :, 1]
img_blue = img[:, :, 2]
```

"only one channel active at a time"

```
only_red = np.zeros(img.shape, dtype=np.uint8)
only_red[:, :, 0] = img_red
only_green = np.zeros(img.shape, dtype=np.uint8)
only_green[:, :, 1] = img_green
only_blue = np.zeros(img.shape, dtype=np.uint8)
only_blue[:, :, 2] = img_blue
```

dtype=np.uint8 NOT optional. Display with nothing.

"arithmetic mean" grayscale

```
img_gray_mean = (img_red.astype(np.float32) + img_green + img_blue) / 3
img_gray_mean = img_gray_mean.astype(np.uint8)
```

.astype first or it wraps.

"weighted luminosity (rgb2gray)"

```
img_gray_weighted = color.rgb2gray(img)
img_gray_weighted = (img_gray_weighted * 255).astype(np.uint8)
```

Returns float [0,1].

"100×100 crop"

```
H, W, C = img.shape
crop = img[0:100, W-100:W, :]      # A: TOP-RIGHT
crop = img[H-100:H, 0:100, :]     # B: BOTTOM-LEFT
```

EX 5 — ResBlock

Track A — classic:

```
class ResBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.conv1 = nn.Conv2d(channels, channels, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(channels)
        self.relu = nn.ReLU()
        self.conv2 = nn.Conv2d(channels, channels, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(channels)

    def forward(self, x):
        identity = x
        out = self.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out = out + identity
        out = self.relu(out)
        return out
```

Track B — bottleneck:

```
class BottleneckResBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        mid = channels // 4
        self.conv1 = nn.Conv2d(channels, mid, kernel_size=1)
        self.bn1 = nn.BatchNorm2d(mid)
        self.conv2 = nn.Conv2d(mid, mid, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(mid)
        self.conv3 = nn.Conv2d(mid, channels, kernel_size=1)
        self.bn3 = nn.BatchNorm2d(channels)
        self.relu = nn.ReLU()

    def forward(self, x):
        identity = x
        out = self.relu(self.bn1(self.conv1(x)))
        out = self.relu(self.bn2(self.conv2(out)))
        out = self.bn3(self.conv3(out))
        out = out + identity
        out = self.relu(out)
        return out
```

ConvBlock — both tracks:

```
class ConvBlock(nn.Module):
    def __init__(self, in_channels):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(in_channels, 64, kernel_size=7, padding='same'),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.Conv2d(64, 64, kernel_size=5, padding='same'),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )
    def forward(self, x):
        return self.block(x)
```

Network — both tracks:

```
class ResNetLike(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.convblock1 = ConvBlock(3)
        self.convblock2 = ConvBlock(64)
        self.resblock1 = ResBlock(64)
        self.resblock2 = ResBlock(64)
        self.flatten = nn.Flatten()
        # 224 -> MaxPool -> 112 -> MaxPool -> 56
        # 64 * 56 * 56 = 200704
        self.fc1 = nn.Linear(200704, 512)
        self.fc2 = nn.Linear(512, 128)
        self.fc3 = nn.Linear(128, num_classes)
```

```
self.relu = nn.ReLU()

def forward(self, x):
    out = self.convblock1(x)
    out = self.convblock2(out)
    out = self.resblock1(out)
    out = self.resblock2(out)
    out = self.flatten(out)
    out = self.relu(self.fc1(out))
    out = self.relu(self.fc2(out))
    out = self.fc3(out)
    return out
```

THE NUMBER: 200704 = $64 \times 56 \times 56$

EX 6 — IDENTICAL BOTH TRACKS

```
def train_one_epoch(model, dataloader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0; n = 0
    for images, labels in dataloader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        running_loss += loss.item(); n += 1
    return running_loss / n

def validate_one_epoch(model, dataloader, criterion, device):
    model.eval()
    running_loss = 0.0; n = 0; correct = 0; total = 0
    with torch.no_grad():
        for images, labels in dataloader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            running_loss += loss.item(); n += 1
            _, predicted = torch.max(outputs, 1)
            correct += (predicted == labels).sum().item()
            total += labels.size(0)
    return running_loss / n, correct / total

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=1e-3, momentum=0.9)

for epoch in range(30):
    train_loss = train_one_epoch(model, train_loader, criterion, optimizer, device)
    val_loss, val_acc = validate_one_epoch(model, val_loader, criterion, device)
    print(f"Epoch [{epoch+1}/30] | train loss: {train_loss:.4f} "
          f"| val loss: {val_loss:.4f} | val acc: {val_acc:.4f}")
```

zero_grad → backward → step. train() vs eval() + no_grad().

DISPLAY BLOCK — copy this shape every time

```
plt.figure()
ax1 = plt.subplot(1, 2, 1)
ax1.imshow(img, cmap="gray", clim=[0, 1])
ax1.set_title("Original")
ax2 = plt.subplot(1, 2, 2, sharex=ax1, sharey=ax1)
ax2.imshow(result, cmap="gray", clim=[0, 1])
ax2.set_title("Result")
plt.savefig(os.path.join(output_root, "fig1.png"))
```

ax.set_title not ax.title. Count the things asked for → that's your subplot grid.

clim — the biggest mark-loser

Array	Display
uint8	cmap="gray", clim=[0,255]
float [0,1]	cmap="gray", clim=[0,1]
gradient / mask / Laplacian	cmap="gray" — NO clim
colour (H,W,3)	NOTHING

Saving

Array	Line
uint8	<code>io.imshow(path, x)</code>
float [0,1]	<code>io.imshow(path, (x*255).astype(np.uint8))</code>
gradient	<code>x = x / x.max()</code> first
mask (signed)	<code>x = (x - x.min())/(x.max()-x.min())</code> first
figure	<code>plt.savefig(path)</code>

Checks (3 seconds each)

```
print(box_filter.sum())    # smoothing -> 1.0
print(sobel_x.sum())      # derivative -> 0.0
print(labels.max())       # watershed -> tens
```

Tomorrow: skeleton → find keyword → type line → display → save → next sub-question.

One sub-question at a time. Never look at the rest of the paper.